서버 운영을 AWS에 맡기면, 우리는 무엇을 설계해야 할까?

서버리스에도 서버는 있고, 운영을 맡길 뿐이다

행사 신청 버튼을 누르면 신청 저장과 참가증 발급, 두 가지 일이 처리된다. 이 기능을 EC2 인스턴스에서 직접 돌린다면 운영체제 패치부터 서버 대수 조정까지 우리가 맡는다.



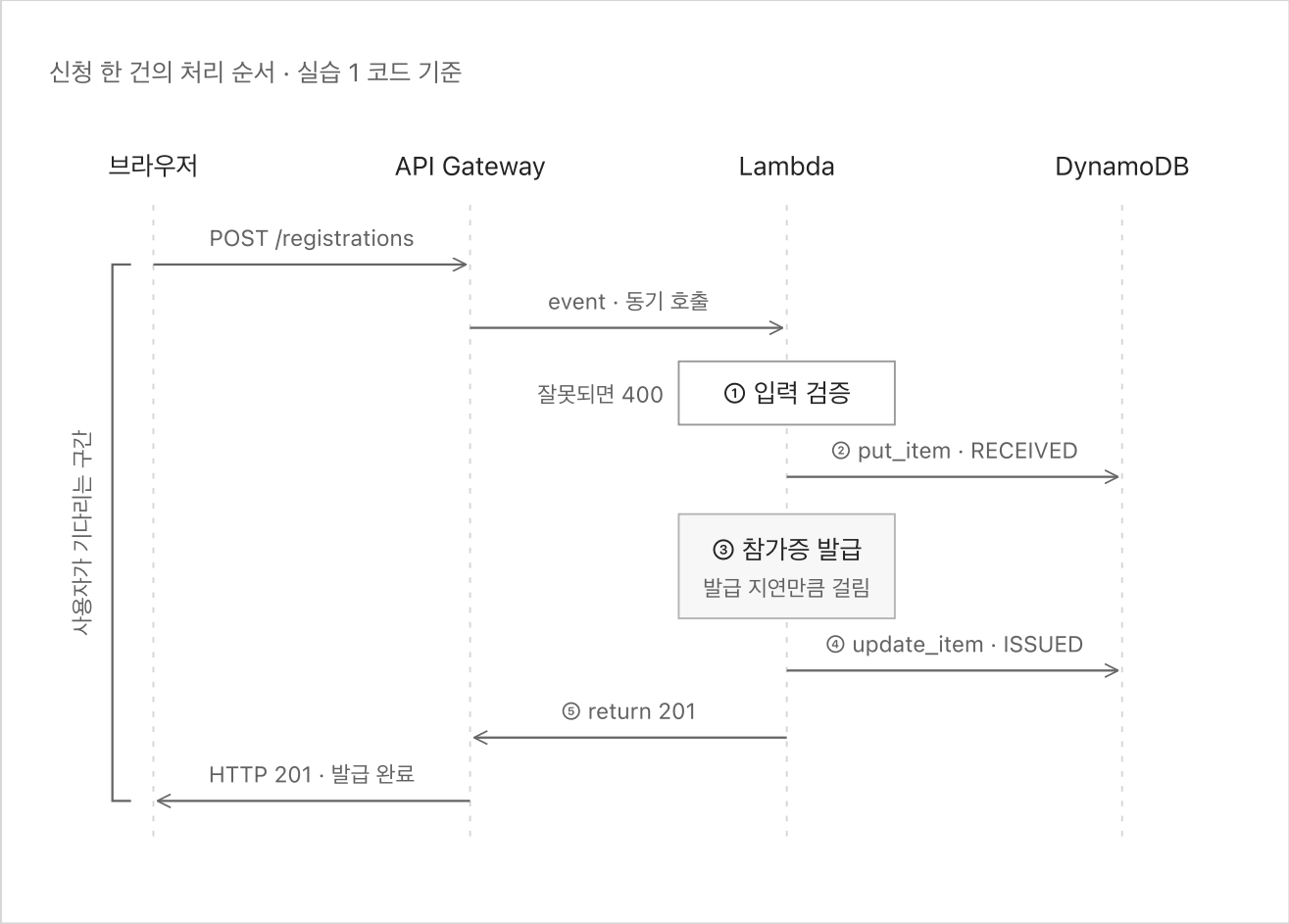
Lambda에서는 AWS가 맡는 층이 위로 올라온다. 코드, 권한, 데이터, 실행 설정은 어느 쪽이든 우리 몫이다.

Lambda를 쓰면 서버 유지보수, 용량 확보, 확장, 패치를 AWS가 관리한다. 서버가 사라진 것이 아니라 **서버 운영의 많은 부분을 AWS에 맡긴 것**이다. 느린 코드가 빨라지거나 데이터 설계가 저절로 되지는 않는다. 요청이 몰리면 실행 환경도 늘어나지만, 기본적으로 한 리전의 모든 함수가 동시 실행 1,000개를 나눠 쓴다.

서버를 누가 운영하는지와 호출한 쪽이 결과를 기다리는지는 다른 기준이다. 이 예제는 서버리스이면서 동기 처리다.

HTTP 요청은 event가 되고, 반환값은 응답이 된다

API Gateway는 HTTP 요청의 메서드, 경로, 헤더, 본문을 JSON 하나로 묶어 Lambda의 handler에 넘긴다. 첫 번째 인자 `event`가 이 요청이고, 두 번째 인자 `context`에는 요청 ID나 남은 실행 시간 같은 실행 정보가 담긴다.



①~⑤는 `register.py`의 handler 안에서 차례로 실행된다. 입력이 잘못되면 ①에서 바로 `400`을 반환한다.

API Gateway는 함수를 동기로 호출하고 반환값을 HTTP 응답으로 바꿔 돌려준다. 그래서 **`return`의 위치가 사용자가 기다리는 범위를 정한다.** 이 코드는 발급 결과까지 기록한 뒤에야 `201`을 반환한다. `event`라는 이름이 비동기를 뜻하지 않듯, `201`도 후속 작업까지 끝났다는 보장은 아니다.

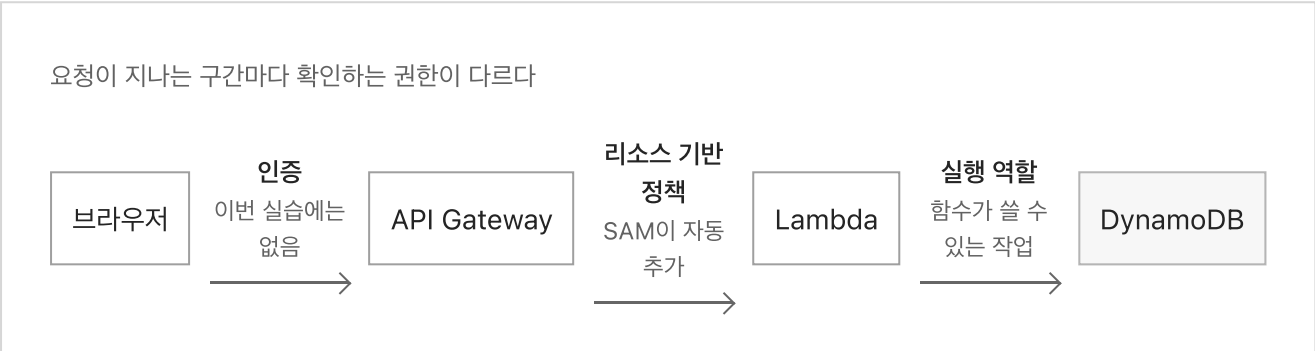
입구에서 요청을 걸러낼 수도 있다. 이번 실습은 CORS만 설정했다.

인증	호출하는 쪽이 API를 써도 되는지 확인한다. HTTP API는 IAM, JWT, Lambda 권한 부여자를 지원한다.
요청량 제한	계정의 리전 단위 한도를 넘는 요청을 `429`로 돌려보낸다.
CORS	브라우저가 다른 출처의 API를 불러도 되는지 정한다. 브라우저가 지키는 규칙이라 인증을 대신하지는 못한다.

함수는 허락받은 일만 할 수 있다

응답에는 결과만 남는다. 처리 과정은 로그에서 본다. 이 코드는 단계마다 신청 번호를 담은 JSON 한 줄을 남기므로, CloudWatch Logs에서 번호로 검색하면 `request_received`부터 `badge_saved`까지 한 번의 실행을 따라갈 수 있다.

그런데 코드에 `put_item`을 적었다고 DB에 쓸 수 있는 것은 아니다. 함수는 실행 역할이라는 IAM 역할에 허용된 AWS 작업만 할 수 있고, 나머지는 `AccessDeniedException`으로 거부된다.



실행 역할은 함수가 다른 AWS 서비스를 쓸 때의 권한이다. 누가 API를 부를 수 있는지는 입구의 인증이 정한다.

이번 템플릿은 신청 함수에 `PutItem`·`UpdateItem`, 상태 조회 함수에 `GetItem`만, 그것도 이 스택의 테이블에만 허용한다. 필요한 작업을 필요한 대상에만 주는 최소 권한이다. 상태 조회 함수가 실수로 신청을 덮어쓰려 해도 실행 역할이 막는다.

실행 환경은 다시 쓰일 수 있지만, 기억을 맡길 곳은 아니다

요청을 처리할 환경이 없으면 Lambda는 코드를 내려받고 런타임을 띄운 뒤 handler 밖의 코드를 한 번 실행한다. 이 초기화로 생기는 지연이 콜드 스타트다. 요청을 마친 환경은 멈춰 두었다가 다음 요청에 다시 쓸 수 있다.



실행 환경 하나는 한 번에 요청 하나만 처리한다. 요청 2는 새로 만든 B가, 요청 3은 그때 비어 있던 A가 받는다.

그래서 `common.py`는 DynamoDB 테이블 객체를 handler 밖에서 만든다. 하지만 다음 요청이 같은 환경으로 온다는 보장은 없다. Lambda는 계속 호출되는 함수의 환경도 몇 시간마다 교체한다. 전역 변수나 `/tmp`에 남긴 신청 결과는 다른 환경에서 보이지 않는다.

> **다시 만들 수 있는 준비는 실행 환경에, 잃으면 안 되는 업무 데이터는 DB에 둔다.**

이렇게 상태를 함수 밖에 두는 것이 상태 외부화다. 걸린 시간은 실행마다 남는 `REPORT` 로그에서 나눠 본다. `Init Duration`은 새 환경의 초기화 시간으로 그 환경의 첫 요청에만 나오고, `Duration`은 handler가 일한 시간이다. 코드에 넣은 발급 지연은 콜드 스타트가 아니라 `Duration`에 들어간다. 이 둘과 별개로 실행 시간 제한이 있고, 이번 템플릿은 10초로 정했다.

어떻게 찾을지 정하고, 그 조회에 맞는 키를 고른다

이 서비스가 가장 자주 묻는 질문은 '이 신청 번호의 상태가 뭐지?'다. 그래서 테이블은 정렬 키 없이 `registrationId` 하나를 기본 키로 쓴다. DynamoDB는 이 파티션 키 값을 해시해 저장할 파티션을 정하고, 읽을 때도 같은 값으로 찾아간다. 키가 정해지면 읽는 방법도 따라 정해진다.

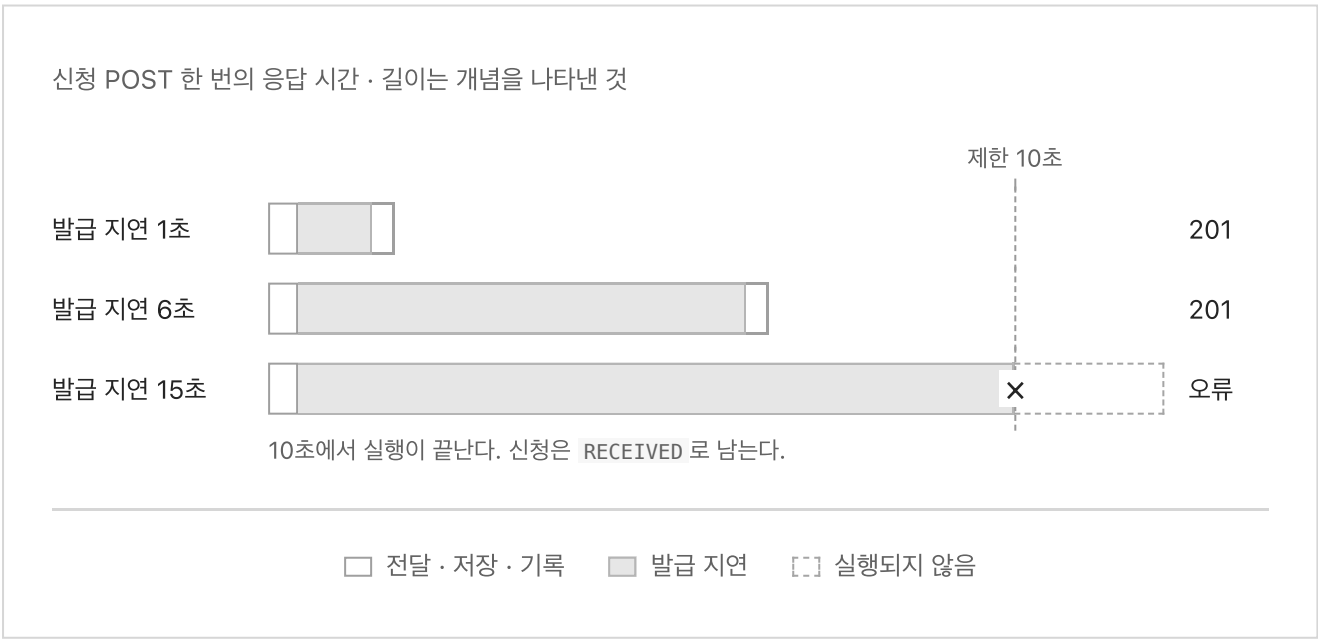
연산	찾는 방식	이 서비스에서
`GetItem`	기본 키 전체로 항목 하나를 읽는다	상태 조회 함수가 쓰는 방식
`Query`	파티션 키가 같은 항목들을 읽고, 정렬 키 조건으로 범위를 좁힌다	정렬 키가 없는 이 테이블에서는 쓰지 않는다
`Scan`	테이블 전체를 읽는다 조건으로 걸러도 읽은 양만큼 용량을 쓴다	`make db`로 작은 실습 테이블을 확인할 때만

상태 조회는 `ConsistentRead=True`로 읽는다. 기본인 최종적 일관성 읽기는 방금 끝난 쓰기를 놓칠 수 있다. 강력한 일관성 읽기는 비용이 두 배지만 신청 직후 상태를 확인하기에 맞다.

이 예제는 온디맨드로 요청한 만큼 과금되고, 프로비저닝이라면 초당 읽기·쓰기 용량을 미리 정한다. 어느 쪽이든 파티션 하나의 처리량은 초당 읽기 3,000, 쓰기 1,000 단위까지라, 요청이 한 키에 몰리면 테이블에 여유가 있어도 그 키에서 제한이 걸린다. 이것이 핫 키 문제다. 신청 번호처럼 값이 다양한 키는 요청이 고르게 퍼진다.

서버를 맡겨도, 사용자는 발급이 끝날 때까지 기다린다

발급 지연을 1초에서 6초로 늘리면 응답도 그만큼 늦어진다. 응답이 발급 뒤에 있기 때문이다. 측정값에는 네트워크와 초기화 시간도 더해지므로 정확히 1초, 6초로 나오지는 않는다.



15초로 넣으면 10초 제한에서 실행이 끊긴다. ②의 저장은 이미 끝났으므로 신청은 `RECEIVED`로 남는다. **오류 응답이 '아무것도 저장되지 않았다'는 뜻은 아니다.** 이때 같은 신청을 다시 보내면 지금 코드는 항목을 덮어쓰고 처음부터 다시 처리한다. 응답이 오지 않을 때의 처리 상태에서 본 문제가 여기서도 생긴다.

제한을 늘려도 HTTP API는 함수의 응답을 최대 30초까지만 기다린다. 느린 처리는 동시성도 붙잡는다. 동시성은 대략 **초당 요청 수 × 평균 처리 시간**이라, 초당 200건이면 1초 처리에 200개, 6초 처리에 1,200개의 실행 환경이 동시에 필요하다. 기본 한도 1,000을 넘는 호출은 Lambda가 거절한다.

접수와 발급을 한 응답에 묶어야 할까?

결과를 지금 받아야 하는 요청이라면 동기 처리가 자연스럽다. 이 서비스에서 판단할 것은 **신청 접수를 알리는 데 참가증 발급 완료까지 필요한가**다.

한 요청 안에서 끝내도 된다	접수와 완료를 나눌 것을 고려
사용자가 결과를 바로 써야 한다 입력 오류 안내, 상태 조회	접수만 확인되면 다음으로 넘어갈 수 있다 신청 접수 → 참가증은 잠시 뒤 확인
처리 시간이 짧고 일정하다	처리 시간이 길거나 들쭉날쭉하다 응답 시간과 동시성이 함께 늘어난다
실패하면 처음부터 다시 요청하면 된다	중간에 실패해도 이미 저장한 신청을 이어서 처리해야 한다 `RECEIVED`로 남은 신청

사용자에게 지금 알려야 하는 것은 '접수됐다'인가, '발급까지 끝났다'인가?

둘을 나누면 접수 시점에 바로 응답할 수 있다. 대신 발급을 어떤 작업이 이어받을지, 같은 신청이 다시 오면 어떻게 알아볼지, 실패하면 언제 다시 처리할지를 정해야 한다. 그때도 기준은 같은 신청 번호다.